

Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs

Anonymous Submission

Author details omitted for double-blind review

Abstract—Speculative decoding is often reported to reduce large language model (LLM) latency, yet gains on consumer GPUs are hardware- and runtime-dependent. We evaluate Qwen2.5 and Qwen3 pairings on RTX4090 and RTX5090-laptop under deterministic and stochastic policies with $k \in \{4, 8, 16\}$. Qwen3 reaches $2.8548\times$ speedup on RTX4090 at deterministic $k=4$, but speedup falls monotonically with k . Under the tested RTX5090-laptop stack, every policy is slower than its autoregressive baseline despite preserving the relative low- k and deterministic preference. Qwen2.5 shows the same RTX4090 policy ranking. We report time-to-first-token (TTFT), time-per-output-token (TPOT), acceptance, sample dispersion, targeted repeat-seed runs, and Holm–Bonferroni-corrected paired tests with effect sizes. The results show that policy ranking is more portable than absolute speedup and that deployment claims require measurement on the target stack.

Index Terms—speculative decoding, large language models, inference acceleration, benchmarking, reproducibility

I. INTRODUCTION

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length- k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [1], [2].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

This study addresses three practical gaps: cross-model and cross-hardware validation, sample- and seed-level variability reporting, and statistical testing against matched autoregressive baselines. The goal is a reproducible, hardware-aware characterization rather than a new decoding algorithm.

A. Research questions

We address four research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on

consumer RTX hardware with Qwen2.5 and Qwen3 pairings?

RQ2. How do draft length and acceptance jointly determine net latency on consumer GPUs?

RQ3. How sensitive is the speed–quality trade-off to deterministic versus stochastic decoding?

RQ4. Do policy rankings and absolute gains transfer from desktop RTX4090 to RTX5090-laptop?

B. Contributions

This paper makes the following contributions:

- 1) A reproducible protocol using frozen manifests and matched prompts on GSM8K, MMLU, and CNN/DailyMail.
- 2) A fixed-policy study of Qwen2.5 and Qwen3 with $k \in \{4, 8, 16\}$ under deterministic and stochastic decoding.
- 3) Cross-hardware validation showing that policy ranking transfers from RTX4090 to RTX5090-laptop while absolute speedup does not.
- 4) Sample-level dispersion, targeted repeat-seed runs, paired tests with Holm–Bonferroni correction, and effect sizes against matched baselines.

Section II reviews prior work; Sections III– V define the protocol, metrics, and implementation; and Sections VI–VII present results, limitations, and conclusions.

II. RELATED WORK

Relevant LLM inference work spans vanilla speculative decoding, architectural methods that reduce drafting cost, diffusion-based drafters, and deployment-oriented systems techniques.

a) *Vanilla speculative decoding*: Leviathan *et al.* [1] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [2] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \quad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \quad (1)$$

where $\tau \in [1, k+1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token

acceptance rate α . Stern *et al.* [3] earlier explored blockwise parallel decoding within a single model.

b) Reducing drafting cost: Medusa [4] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one pass. The EAGLE family [5]–[7] retains an external draft conditioned on target hidden features. These methods reduce drafting cost but remain sensitive to model, runtime, and serving conditions, motivating end-to-end measurement on deployment hardware.

c) Diffusion-based drafters: Recent work directly attacks the linear-in- k drafting bottleneck. DiffuSpec [8] and SpecDiff-2 [9] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [10] focuses on a low-cost parallel draft model adaptation. DFlash [11] uses a lightweight 5-layer block-diffusion drafter conditioned on target hidden features and reports up to $5\times$ speedup over autoregressive baselines in real-world serving scenarios on B200 hardware (over $6\times$ under idealized H200 decoding benchmarks).

d) Cascaded speculative control: Recent cascaded architectures dynamically balance inference cost and generation quality. CAS-Spec builds an on-the-fly hierarchy of draft stages from a single target model using Dynamically Switchable Inference Acceleration (DSIA) strategies (for example, layer sparsity and activation quantization), coordinated by a Dynamic Tree Cascade (DyTC) algorithm [12]. Speculative Cascades applies a deferral rule, invoking larger models only for hard inputs while routing easier cases to smaller models [13]. Although these methods report strong acceleration, they typically require target-side changes, additional training, or complex serving stacks. Our study instead evaluates unmodified, same-family draft and target models through a conventional draft–verify loop.

e) Precision and systems deployment: Deployment-oriented inference work shows that quantization and memory traffic are central to practical throughput gains on constrained hardware. Low-precision techniques such as LLM.int8() [14] reduce weight movement and can interact with speculative decoding in two opposing ways: lower per-step cost for draft/verify passes, but possible shifts in acceptance behavior when verifier and drafter precision settings differ. This interaction motivates reporting speed and quality jointly, rather than throughput alone.

f) Distributed and edge relevance: In distributed and mobile computing settings, local inference can reduce round-trip latency and cloud dependency for interactive workloads. In this context, speculative decoding is relevant beyond chatbot serving, including edge language interfaces where bounded latency and predictable runtime matter as much as peak throughput.

g) Position of this work: This work evaluates vanilla speculative decoding with Qwen2.5 and Qwen3 same-family pairings under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, it char-

acterizes acceptance, quality, and latency across desktop RTX4090 and RTX5090-laptop profiles.

III. EXPERIMENTAL PROTOCOL

A. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

- 1) **GSM8K** [15] test subset: 300 samples.
- 2) **MMLU** [16] subset: 500 samples ($5 \text{ subjects} \times 100 \text{ each}$).
- 3) **CNN/DailyMail** [17] subset: 200 samples.

Sampling policy: Primary benchmarking uses fixed random seed = 42, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. To assess run-to-run stability, we additionally repeated the RTX4090 and RTX5090-laptop $k=4$ settings (deterministic and stochastic) at seeds 123 and 999 for both Qwen2.5 (draft 0.5B) and Qwen3 (draft 0.6B) on the same manifests. Each output CSV records seed and sample identifier, enabling paired analysis at sample and seed levels where repeats are available.

B. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
Question: {question}

MMLU:

You are a knowledgeable assistant. Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:

CNN/DailyMail:

Summarize the following article in 3–4 sentences, preserving key facts and entities.
Article: {article}
Summary:

a) Objective and hypotheses: This study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from $k=4$ to $k=8$ improves speedup, while $k=16$ may reduce gains due to additional drafting and verification overhead.
- **H3:** Increasing k tends to improve accepted block size but can reduce end-to-end speed once drafting/verification overhead dominates.
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k .

b) *Hardware and software controls*: We report two consumer-GPU run families: an NVIDIA RTX4090 desktop profile and an RTX5090-laptop profile. Both use the same Python runtime entry points, prompt templates, tokenizer settings, and serial execution policy to minimize cross-run contention. We keep software dependencies pinned within each run family and preserve frozen manifests for paired comparison (Table I).

c) *Runtime setup*: We keep fixed step shapes, pre-allocate static draft/verify buffers, and warm up the inference path before measurement. During decoding, each cycle updates buffer contents under shape guards and falls back to eager execution when required. This preserves output semantics and reduces avoidable runtime-path variance in short speculative cycles.

d) *Model pairing*: We evaluate two same-family pairings. For Qwen2.5, the target is Qwen2.5-3B-Instruct with Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct drafts on the RTX4090 run family, with the 0.5B $k=4$ stability setting also repeated on RTX5090-laptop. For Qwen3, the target is Qwen3-4B-Instruct with a Qwen3-0.6B-Instruct draft on RTX4090 and RTX5090-laptop run families. For each family, fixed-policy experiments sweep $k \in \{4, 8, 16\}$ under deterministic/stochastic regimes with regime-matched autoregressive baselines. All configurations process fixed manifests to preserve paired comparability.

e) *Decoding regimes and grid*: Two regimes are evaluated:

- **Deterministic**: greedy decoding ($T=0.0$, $\text{top-}p=1.0$; sampling disabled).
- **Stochastic**: sampling with $T=0.7$, $\text{top-}p=0.9$, and no top- k truncation.

For the primary fixed-policy matrix, we sweep $k \in \{4, 8, 16\}$ under both regimes (6 configurations), plus regime-matched autoregressive baselines.

f) *Endpoints*: The primary endpoint is paired wall-clock speedup $S = L_{\text{autoregressive}}/L_{\text{spec}}$. Table II defines secondary runtime, acceptance, and quality endpoints. We report sample-level mean $\pm$ std and one-sided paired t -tests with Holm-Bonferroni correction and Cohen’s d against regime-matched baselines. Repeat-seed summaries are limited to $k=4$ for both families and devices.

IV. EVALUATION METRICS

Table II defines the exact reporting metrics used in this study.

A. Success criteria

Primary:

- 1) Mean speedup $S \geq 1.3\times$ with Holm-Bonferroni-corrected $p_S < 0.05$.
- 2) Absolute quality drop ≤ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify the best (k , regime) maximizing S under quality constraints, report sample-level dispersion (mean $\pm$ std), report seed-level stability where repeated seeds are available,

and test whether ranking trends transfer across hardware profiles even when absolute speedups differ.

V. IMPLEMENTATION AND REPRODUCIBILITY

a) *Code organization*: The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, and autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

b) *Model loading and precision*: In the current configuration, target and draft models are loaded in 16-bit floating point (FP16) for the main sweep, with quantization controls retained in configuration for alternative runs. The tokenizer and generation limits are kept constant between baselines and speculative runs to preserve paired comparability.

c) *Baselines*: We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

d) *Speculative decoding implementation*: The speculative loop follows standard token-level draft-verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Both draft and target states are tracked with key-value (KV) caches throughout the decode loop. Figure 1 summarizes this implementation flow.

e) *CUDA-graph runtime note*: Graph-capture metadata differs by run family. The Qwen3 RTX4090 summary records captured execution with capture rate 1.0, whereas the Qwen2.5 RTX4090 summary records pending capture with rate 0.0. RTX5090-laptop sensitivity runs do not show a consistent latency benefit from graph-enabled toggles. CUDA graphs are therefore reported as runtime context, not as an independently evaluated optimization.

f) *Dynamic control flow*: Acceptance-dependent branches and cache updates require host-visible orchestration in the native PyTorch loop and can prevent full graph capture. Because this overhead was not isolated experimentally, we treat it as a possible explanation for runtime sensitivity rather than a measured cause.

g) *Tokenizer scope*: Same-family pairings use matching tokenizers and avoid vocabulary alignment inside verification. Cross-tokenizer pairings are outside this study; their mapping overhead and acceptance behavior require separate evaluation.

h) *Regimes and outputs*: The same verification core is used for deterministic and stochastic modes, with only the sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

TABLE I
CONSUMER-GPU RUN-FAMILY PROFILES.

Component	RTX4090 desktop	RTX5090 laptop
GPU memory	24 GB GDDR6X	24 GB GDDR7
Memory bandwidth	1008 GB/s	896 GB/s
CUDA cores	16384	10496
Architecture	Ada Lovelace	Blackwell
Power profile	450 W TGP class	Up to 150 W TGP (OEM-dependent)
Software stack	PyTorch + Transformers (pinned per run family)	

TABLE II
METRIC DEFINITIONS USED IN THIS STUDY.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail ROUGE-L	Q_{sum}	standard ROUGE-L F1 (rouge-score, stemmed)	%	Higher
CNN/DailyMail BLEU	Q_{bleu}	sentence-level BLEU (NLTK, method1 smoothing)	%	Higher
CNN/DailyMail BERTScore	Q_{bert}	BERTScore F1 (DistilBERT-base-uncased)	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or +
Speedup significance	p_S	one-sided paired t -test for $H_0 : S \leq 1$, Holm-Bonferroni corrected	p-value	Lower
Speedup effect size	d_S	Cohen’s d for paired baseline/speculative latency	σ units	Higher

VI. RESULTS

This section reports fixed-policy results for Qwen2.5 and Qwen3 families across two consumer-GPU setups: desktop RTX4090 and RTX5090-laptop. We focus on paired speedup S , acceptance dynamics (α , B_{eff}), and token-timing metrics (time to first token, TTFT; time per output token, TPOT), then summarize portability implications.

A. RQ1: Does speculative decoding accelerate inference?

On RTX4090, all six Qwen2.5 settings and five of six Qwen3 settings exceed baseline (Tables III and IV). Both families peak at deterministic $k=4$ (Qwen2.5: $3.0674\times$; Qwen3: $2.8548\times$). Qwen3 stochastic $k=16$ is the exception ($S=0.9719$), showing that long proposals can eliminate the gain.

B. RQ2: How do k and acceptance affect net latency?

Larger k increases B_{eff} but reduces end-to-end speed, the overhead inflection predicted by Eq. 1. Mean speedup falls from 2.9164 to 1.8118 for Qwen2.5 and from 2.6673 to 1.0405 for Qwen3 between $k=4$ and $k=16$. Thus, additional drafting and verification cost dominates the accepted-block benefit in this range.

C. RQ3: Deterministic versus stochastic regime

Qwen3 deterministic decoding is faster at every matched k , with its advantage narrowing from 0.3750 at $k=4$ to 0.1372 at $k=16$. Qwen2.5 follows the same pattern at $k=4$ and 8, with a small reversal at $k=16$. Deterministic decoding therefore

provides the stronger low- k runtime profile, although the preference is not universal.

D. RQ4: Cross-hardware portability (RTX4090 vs RTX5090-laptop)

Cross-hardware validation (Table V) reverses the practical outcome: all six RTX5090-laptop settings are slower than their autoregressive baselines ($S=0.2222\text{--}0.6743$). Relative ordering remains consistent—low k and deterministic decoding perform best—but RTX4090 speedups do not transfer. Deployment decisions must therefore be based on target-stack measurements, not GPU generation or policy ranking alone.

Figure 2 consolidates the expanded comparison across model families, hardware profiles, and repeat seeds.

E. Variability, token timing, and statistical tests

For Qwen3 RTX4090, per-sample speedup (mean $\pm$ std, $n=1000$) decreases from 2.7561 ± 0.5950 to 1.3549 ± 0.6533 across deterministic $k=4\text{--}16$, and from 2.4945 ± 0.4565 to 1.2173 ± 0.6206 under stochastic decoding. Over the same range, mean TTFT rises from 132.57 to 349.58 ms (deterministic) and 134.54 to 354.82 ms (stochastic); TPOT rises from 41.23 to 94.24 ms and 45.51 to 106.03 ms, respectively. Longer proposals therefore degrade both initial and streaming responsiveness.

Seed stability was evaluated only at $k=4$ using the same manifests ($n=1000$) for seeds 123 and 999. Figure 2(d) summarizes the change in mean latency: deterministic runs differ by at most 1.5%, while stochastic differences range from

TABLE III
QWEN2.5 FIXED-POLICY RESULTS ON RTX4090 (TARGET: 3B, DRAFT: 0.5B) AGAINST REGIME-MATCHED AUTOREGRESSIVE BASELINES. SPEEDUP IS COMPUTED AS A RATIO OF MEANS AGAINST THE CORRESPONDING BASELINE.

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
0.5B, $k=4$, det	3.0674	0.4212	1.66	3.7210	102.35	29.06
0.5B, $k=4$, stoch	2.9394	0.4080	1.61	3.9631	105.26	30.89
0.5B, $k=8$, det	2.4542	0.3515	2.72	4.6508	162.76	36.73
0.5B, $k=8$, stoch	2.4063	0.3444	2.67	4.8411	165.95	38.04
0.5B, $k=16$, det	1.8050	0.2539	3.76	6.3234	277.43	49.34
0.5B, $k=16$, stoch	1.8185	0.2516	3.73	6.4059	278.85	49.81

TABLE IV
QWEN3 FIXED-POLICY RESULTS ON RTX4090 AGAINST REGIME-MATCHED AUTOREGRESSIVE BASELINES. SPEEDUP IS COMPUTED AS A RATIO OF MEANS AGAINST THE CORRESPONDING BASELINE.

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
0.6B, $k=4$, det	2.8548	0.3362	1.33	4.8881	132.57	41.23
0.6B, $k=4$, stoch	2.4798	0.2672	1.06	5.6196	134.54	45.51
0.6B, $k=8$, det	1.9648	0.2378	1.86	7.1024	204.99	58.76
0.6B, $k=8$, stoch	1.6869	0.1916	1.50	8.2612	207.82	64.67
0.6B, $k=16$, det	1.1091	0.1292	1.96	12.5817	349.58	94.24
0.6B, $k=16$, stoch	0.9719	0.1082	1.64	14.3390	354.82	106.03

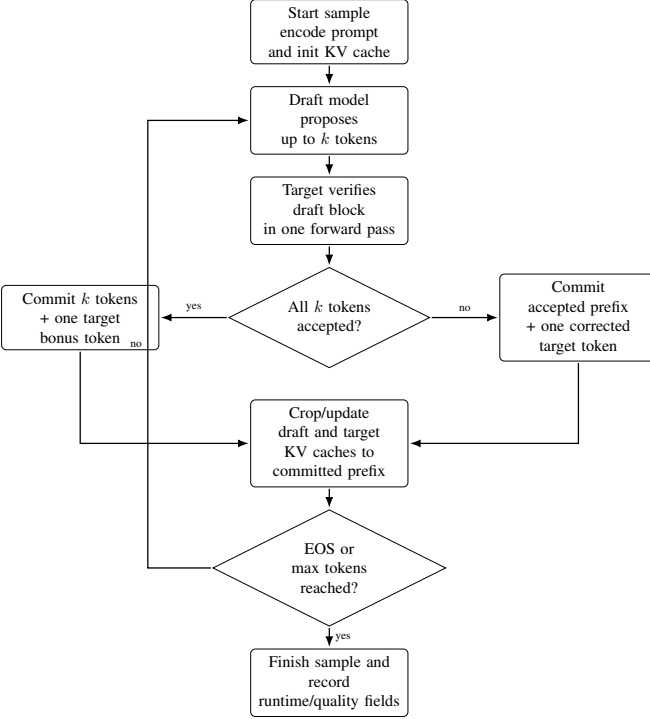


Fig. 1. Vanilla speculative decoding implementation flow. The target model remains the verifier of record; accepted draft prefixes are committed, and caches are cropped to preserve exact target-consistent decoding.

0.4% to 5.0% across the two families and devices. Regime ordering is preserved, and acceptance and token timing vary less across seeds than across prompts. These repeats do not establish seed stability for $k=8$ or 16.

One-sided paired t -tests ($H_0: S \leq 1$), paired by sample and corrected across six Qwen3 RTX4090 configurations using Holm–Bonferroni, confirm speedup for deterministic $k=4$, 8,

TABLE V
QWEN3 FIXED-POLICY RESULTS ON RTX5090-LAPTOP. ORDERING TRENDS ARE PRESERVED (BEST AT LOW k , DETERMINISTIC > STOCHASTIC), BUT ABSOLUTE SPEEDUPS ARE BELOW 1 UNDER THE TESTED RUNTIME STACK.

Config	S	α	B_{eff}
0.6B, $k=4$, det	0.6743	0.3353	1.32
0.6B, $k=4$, stoch	0.5924	0.2845	1.06
0.6B, $k=8$, det	0.4613	0.2538	1.85
0.6B, $k=8$, stoch	0.3995	0.2273	1.50
0.6B, $k=16$, det	0.2588	0.1803	1.96
0.6B, $k=16$, stoch	0.2222	0.1543	1.64

and 16 ($d_S=1.18$, 1.10, and 0.29; all $p_S < 10^{-18}$) and stochastic $k=4$ and 8 ($d_S=1.24$ and 1.13; both $p_S < 10^{-179}$). Stochastic $k=16$ is not significant ($d_S = -0.08$, $p_S=0.996$), consistent with its below-baseline mean. Tests use the seed-42 full grid; repeat seeds cover $k=4$ only.

CNN/DailyMail quality also changes under deterministic speculative decoding ($n=200$). Relative to the autoregressive baseline ($Q_{\text{sum}}=19.58$, $Q_{\text{bleu}}=3.20$, $Q_{\text{bert}}=0.793$), $k=4$ yields 11.50, 1.39, and 0.710; $k=16$ partially recovers to 14.92, 2.04, and 0.741. All three metrics therefore identify a speed–quality trade-off rather than free acceleration.

F. Hypothesis resolution

H1 is supported for all Qwen2.5 and five of six Qwen3 RTX4090 settings, but rejected for the RTX5090-laptop matrix. H2 is rejected: increasing k from 4 to 8 reduces speedup. H3 is supported because B_{eff} grows while latency worsens. H4 is mostly supported: deterministic decoding leads at all Qwen3 points and at low-to-mid k for Qwen2.5, but Qwen2.5 $k=16$ shows a small reversal. These outcomes bound the claims to the evaluated model, hardware, and runtime combinations.

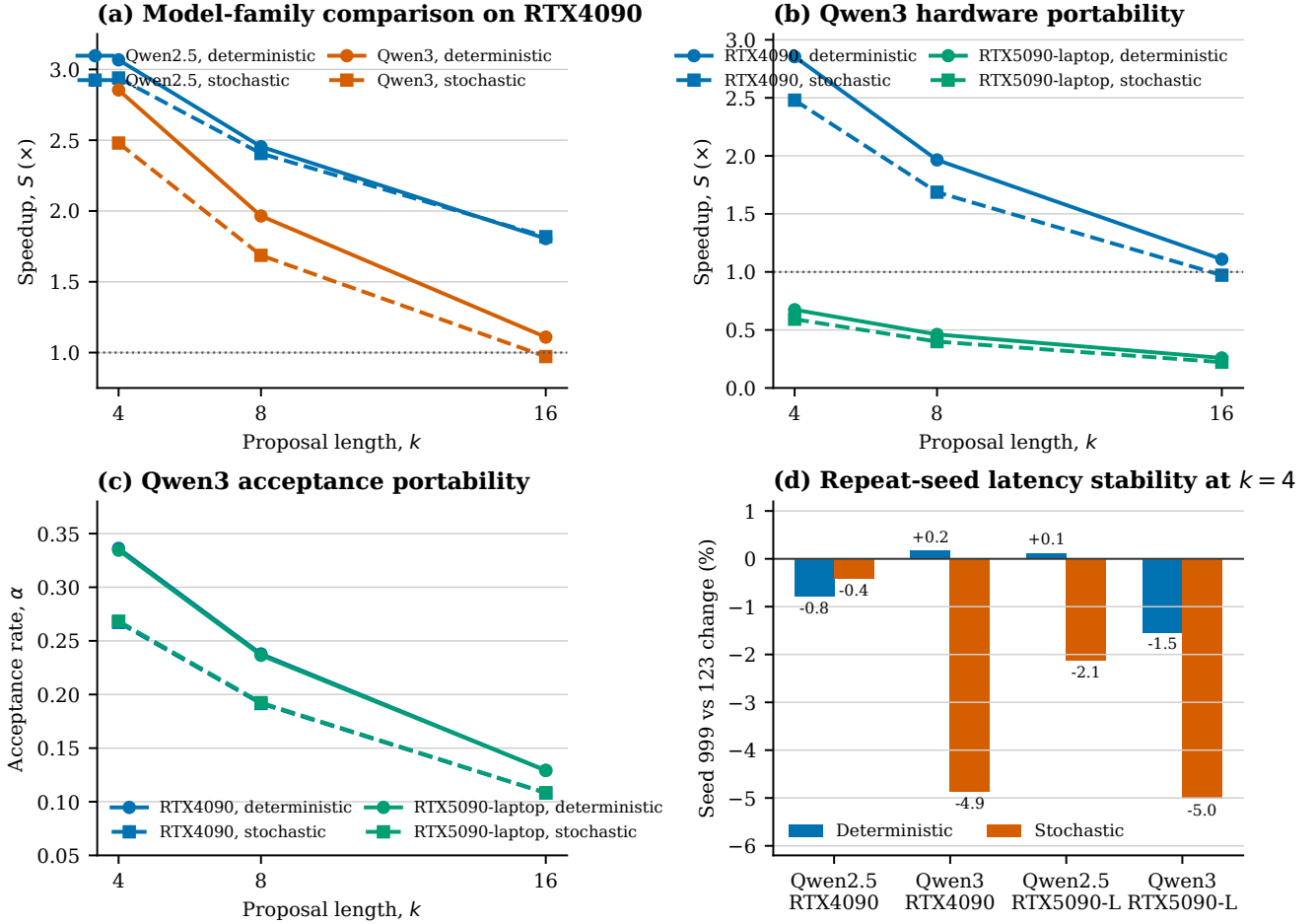


Fig. 2. Expanded fixed-policy and stability comparison. (a) Qwen2.5 and Qwen3 speedup on RTX4090; (b) Qwen3 speedup on RTX4090 and RTX5090-laptop; (c) Qwen3 acceptance-rate portability; and (d) the change in mean latency from repeat seed 123 to seed 999 at $k=4$ for both model families and hardware profiles. Panels (a)–(c) use the primary seed-42 runs and regime-matched autoregressive baselines. The dotted line in (a)–(b) marks baseline parity ($S=1$).

VII. DISCUSSION

A. Main empirical takeaways

The evaluated systems support two deployment rules. First, low proposal length is preferred: increasing k raises accepted block size but worsens TTFT, TPOT, and total latency. Second, deterministic decoding is usually faster than stochastic decoding at matched k . These rankings recur across model families and devices, but their outcomes differ: RTX4090 achieves net acceleration, whereas every RTX5090-laptop configuration is subbaseline. Reported speedups should therefore be treated as properties of a complete model–hardware–runtime stack, not of a GPU generation or policy alone.

B. Novelty and contribution scope

The contribution is empirical and systems-oriented, not a new decoding algorithm. Frozen manifests, matched baselines, raw artifacts, and paired statistics connect acceptance and token timing to practical operating choices on two consumer GPU profiles.

C. Variance and evaluation depth

The corrected paired tests distinguish five significant Qwen3 RTX4090 gains from the nonsignificant stochastic $k=16$ result. Repeat-seed evidence is more limited: seeds 123 and 999 cover only $k=4$, although they preserve regime ordering across both families and devices. CNN/DailyMail ROUGE-L, BLEU, and BERTScore jointly reveal a quality cost that partially recovers as k increases, opposing the speed trend. These automated metrics do not fully measure factuality or task utility.

VIII. THREATS TO VALIDITY

Hardware and runtime dependence. The tested PyTorch/Transformers stack accelerates Qwen3 on RTX4090 but not on RTX5090-laptop. Results cannot be extrapolated to other consumer or data-center GPUs without target-stack measurement.

Model-family scope. This study centers on same-family Qwen2.5 and Qwen3 target/draft pairings. Cross-family spec-

ulative pairings may show different acceptance and quality behavior.

Evaluation breadth. The benchmark suite covers reasoning, knowledge QA, and summarization, but does not exhaust long-context, safety, or factual consistency requirements.

Sample-level versus seed-level dispersion. Mean $\pm$ std values describe prompt dispersion within a seed. Repeat seeds cover only $k=4$; full-grid multi-seed evidence remains future work.

IX. CONCLUSION

Speculative decoding on consumer GPUs is strongly stack-dependent. Deterministic $k=4$ is best on RTX4090, but the same policy family remains slower than autoregressive decoding on RTX5090-laptop. Target-hardware benchmarking is therefore required before deployment. Future work should repeat the full grid across seeds, broaden model and hardware coverage, and evaluate longer contexts and factuality-sensitive quality metrics.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” in *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.01318>
- [3] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise parallel decoding for deep autoregressive models,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. [Online]. Available: <https://doi.org/10.48550/arXiv.1811.03115>
- [4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.10774>
- [5] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.15077>
- [6] —, “EAGLE-2: Faster inference of language models with dynamic draft trees,” *arXiv preprint arXiv:2406.16858*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.16858>
- [7] —, “EAGLE-3: Scaling up inference acceleration of large language models via training-time test,” *arXiv preprint arXiv:2503.01840*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.01840>
- [8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, “DiffuSpec: Unlocking diffusion language models for speculative decoding,” *arXiv preprint arXiv:2510.02358v1*, 2025, <https://arxiv.org/abs/2510.02358v1>. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.02358>
- [9] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto, “SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding,” *arXiv preprint arXiv:2511.00606*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.00606>
- [10] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum, “PARD: Accelerating llm inference with low-cost parallel draft model adaptation,” *arXiv preprint arXiv:2504.18583*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2504.18583>
- [11] J. Chen, Y. Liang, and Z. Liu, “DFlash: Block diffusion for flash speculative decoding,” *arXiv preprint arXiv:2602.06036*, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.06036>
- [12] Z. Ning, J. Shao, R. Xu, X. Guo, J. Zhang, C. Zhang, and X. Li, “CAS-Spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms,” *arXiv preprint arXiv:2510.26843*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.26843>
- [13] H. Narasimhan, W. Jitkrittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, “Faster cascades via speculative decoding,” in *Proceedings of the IEEE Conference (arXiv preprint)*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.19261>
- [14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2208.07339>
- [15] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2110.14168>
- [16] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2009.03300>
- [17] A. See, P. J. Liu, and C. D. Manning, “Get to the point: Summarization with pointer-generator networks,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1704.04368>